Distributed Digit Relay (Turtlesim)

Laporan Cara Kerja dan Penjelasan Kode
Tugas Kelompok ROS2

Nama	NRP
Rafi Attar Maulana	5025241141
I Made Ryan Darmajaya	5025241158
Hosea Felix Sanjaya	5025241177
Kadek Andra Wikanjaya Putra	5025241187
M. Hilman Azhar	5025241264
A. Wildan Kevin Assyauqi	5025241265

1. Gambaran Umum

Sistem ini terdiri dari enam node anggota yang menggambar angka 1 sampai 12 secara bergantian di `turtlesim`, dalam dua siklus. Tidak ada node pusat: giliran diatur lewat satu topic, yaitu `/relay/turn`, dan node yang bukan gilirannya tetap diam (idle).

Kode dibagi menjadi dua package:

Package	Isi
`relay_interfaces`	Definisi pesan `Turn.msg` (`ament_cmake`).
`digit_relay`	Modul bersama `common/`, enam node `member_1` sampai `member_6`, dan launch file (`ament_python`).

Pembagian angka ke anggota mengikuti rumus `member_of(n) = (n - 1) % 6 + 1`:

Angka (siklus 1)	1	2	3	4	5	6
Angka (siklus 2)	7	8	9	10	11	12
Anggota	1	2	3	4	5	6

2. Alur Kerja

1. `ros2 launch digit_relay relay.launch.py` menjalankan `turtlesim_node` dan keenam node anggota.
2. Tiap node menunggu service dan pose turtlesim siap (pengecekan berkala, tanpa delay tetap).
3. `member_1` mengirim pesan giliran pertama: angka 1, anggota 1.
4. Semua node menerima pesan. Hanya node dengan `active_member` yang cocok yang menggambar; lainnya diam.
5. Setelah selesai menggambar, node mengirim pesan giliran berikutnya (angka + 1, anggota berikutnya).
6. Sebelum menggambar angka 7, `member_1` memanggil service `/clear` untuk membersihkan kanvas (awal siklus 2).
7. Setelah angka 12 selesai, `member_6` mengirim pesan dengan `finished = true`. Semua node kembali idle.

3. Penjelasan Kode

3.1. Pesan giliran (`Turn.msg`)

```
int32 current_number
int32 active_member
bool finished
```

Tiga field: angka yang digambar sekarang, anggota yang aktif, dan penanda relay selesai. Nilai `active_member = 0` berarti tidak ada anggota aktif.

3.2. `common/config.py`

```
def member_of(number):
    return (number - 1) % NUM_MEMBERS + 1
```

Operator `%` adalah sisa bagi. Untuk angka 7: $(7-1) \bmod 6 + 1 = 1$, jadi angka 7 kembali ke anggota 1. File ini juga menyimpan konstanta bersama seperti `NUM_MEMBERS`, `TOTAL_NUMBERS`, dan nama topic/service.

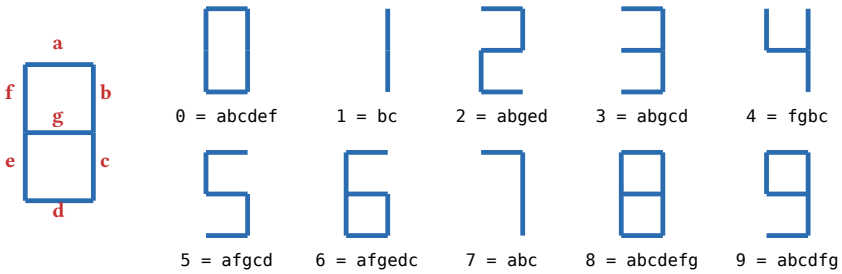
3.3. `common/glyphs.py`: bentuk digit

```
'7': [[(0, 1), (1, 1), (0.4, 0)]],
```

Setiap digit adalah daftar goresan, dan setiap goresan adalah daftar titik `(x, y)` pada kotak 0 sampai 1. Digit 7 punya satu goresan: kiri-atas, kanan-atas, lalu turun ke bawah. Fungsi `digit_strokes` menskalakan titik ke ukuran sebenarnya:

```
[(x0 + px * width, y0 + py * height) for px, py in stroke]
```

Ini *list comprehension*: untuk tiap titik `(px, py)` hitung koordinat kanvas, lalu kumpulkan jadi daftar baru.



Gambar 1: Kiri: nama tiap segmen. Kanan: segmen yang menyala untuk angka 0–9.

3.4. `common/layout.py`: posisi slot

Kanvas dibagi grid 2 baris dan 3 kolom (slot 1 sampai 3 di atas, 4 sampai 6 di bawah).

```
row = (member_id - 1) // COLUMNS
col = (member_id - 1) % COLUMNS
```

Operator `//` adalah pembagian bulat, dipakai untuk mencari baris. Angka dua digit (10, 11, 12) diletakkan di tengah slot dengan jarak antar-digit `DIGIT_GAP`, sehingga tidak menabrak slot tetangga maupun dinding. `strokes_for_number` mengembalikan semua goresan angka dalam koordinat kanvas.

3.5. `common/turtle_driver.py`: menggerakkan turtle

Driver menyimpan antrean perintah (`pen` untuk mengatur pena, `goto` untuk berpindah) dan menjalankannya satu per satu dengan timer:

```
self._timer = node.create_timer(tick, self._on_tick)
future = self._teleport_client.call_async(req)
```

- `call_async` memanggil service tanpa menunggu hasilnya (non-blocking), sehingga node tidak membeku. `future` dicek di tick berikutnya dengan `future.done()`.
- `goto` bergerak bertahap: tiap tick maju sejauh `step`, jadi gerakan terlihat halus. `draw_step` untuk menggambar dan `travel_step` (lebih besar) untuk pindah slot.
- Setelah tiap teleport, driver menunggu `/turtle1/pose` sampai posisinya sama dengan tujuan. Dengan begitu garis sudah tergambar sebelum pena diubah.
- Di `draw_polylines`, pena diangkat (`off = 1`) saat menuju titik awal goresan, lalu diturunkan (`off = 0`) saat menggambar.

3.6. `common/base_member.py`: kerangka node

Kelas `BaseMember` adalah turunan `Node` yang dipakai semua anggota. Bagian terpenting:

```
qos = QoSProfile(depth=1,
                 reliability=ReliabilityPolicy.RELIABLE,
                 durability=DurabilityPolicy.TRANSIENT_LOCAL)
self.create_subscription(Turn, TURN_TOPIC, self._on_turn, qos)
```

`TRANSIENT_LOCAL` membuat pesan terakhir disimpan, sehingga node yang start belakangan tetap menerima giliran. Inilah yang membuat sinkronisasi berjalan tanpa delay manual.

```
if msg.active_member != self.member_id:
    return
```

Baris ini membuat node tetap idle jika bukan gilirannya. Pesan lama atau duplikat diabaikan lewat `_last_number`.

`BaseMember` menyediakan tiga *hook* yang bisa ditimpa (*override*) oleh anggota: `on_ready`, `before_draw`, dan `on_last_number`. Pengaturannya lewat pewarisan kelas.

3.7. `member_1.py` sampai `member_6.py`

```
class Member2(BaseMember):
    def __init__(self):
        super().__init__(member_id=2, pen_rgb=(255, 150, 0), pen_width=3)
```

Tiap anggota mewarisi `BaseMember` dan memanggil `super().__init__` untuk mengisi ID dan warna pena. Dua anggota punya tugas tambahan:

- `member_1` menimpa `on_ready` (mengirim giliran pertama) dan `before_draw` (memanggil `/clear` jika `(number - 1) % NUM_MEMBERS == 0` dan `number > 1`, yaitu awal siklus baru).
- `member_6` menimpa `on_last_number` untuk mengirim pesan `finished = true`.

3.8. `launch/relay.launch.py` dan `setup.py`

```
for member_id in range(1, NUM_MEMBERS + 1):
    nodes.append(Node(package='digit_relay',
                      executable=f'member_{member_id}',
                      name=f'member_{member_id}'))
```

Perulangan ini membuat enam objek `Node`. Kata `f'...'` adalah *f-string* yang menyisipkan nilai variabel ke dalam teks. Nama executable didaftarkan di `setup.py` pada bagian `entry_points`, misalnya `'member_1 = digit_relay.member_1:main'`, yang berarti perintah `member_1` menjalankan fungsi `main` di file `member_1.py`.

4. Cara Menjalankan

```
cd ~/ros2_ws
colcon build
source install/setup.bash
ros2 launch digit_relay relay.launch.py
```

5. Parameter yang Dapat Disesuaikan

Parameter	Fungsi
`draw_step`, `travel_step`	Kecepatan menggambar dan pindah slot (`turtle_driver.py`).
`DIGIT_W`, `DIGIT_H`, `DIGIT_GAP`, `MARGIN`	Ukuran digit dan jarak aman dari dinding (`layout.py`).
`pen_rgb`, `pen_width`	Warna dan ketebalan pena tiap anggota (`member_N.py`).